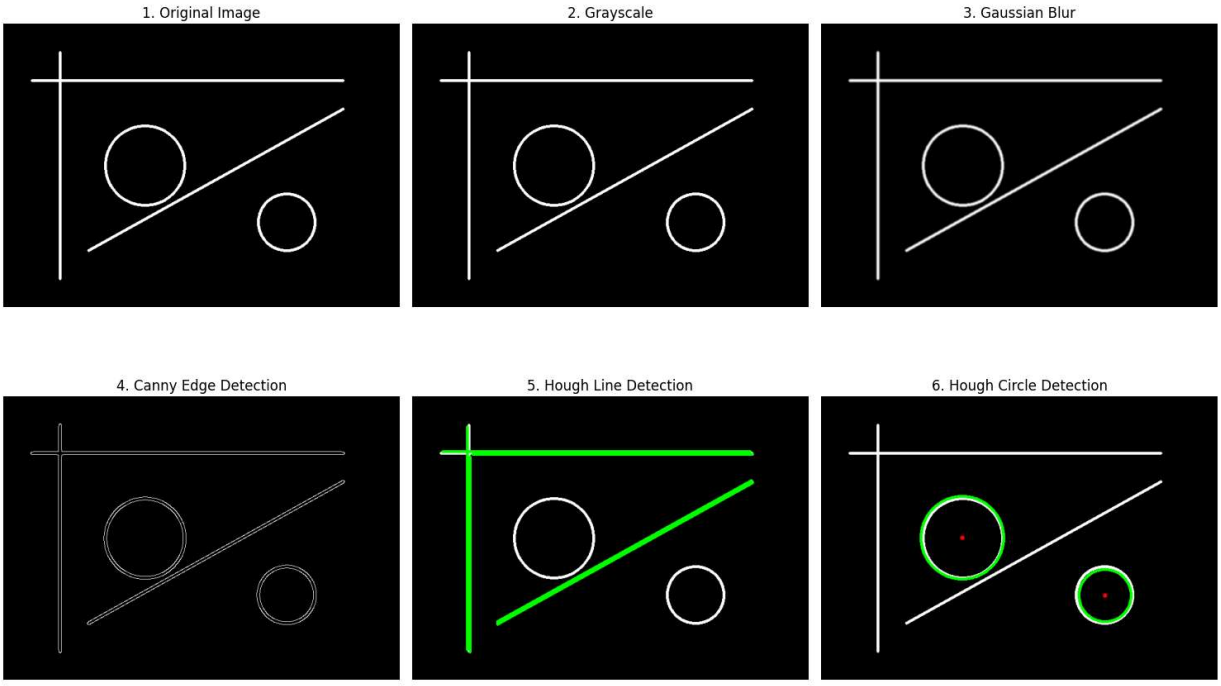
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# ------------------------------------------------------------
# STEP 1: CREATE A SAMPLE IMAGE
# ------------------------------------------------------------

img = np.zeros((500, 700, 3), dtype=np.uint8)

# Draw lines
cv2.line(img, (50, 100), (600, 100), (255, 255, 255), 3)
cv2.line(img, (100, 50), (100, 450), (255, 255, 255), 3)
cv2.line(img, (150, 400), (600, 150), (255, 255, 255), 3)

# Draw circles
cv2.circle(img, (250, 250), 70, (255, 255, 255), 3)
cv2.circle(img, (500, 350), 50, (255, 255, 255), 3)

# ------------------------------------------------------------
# STEP 2: CONVERT IMAGE TO GRAYSCALE
# ------------------------------------------------------------

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ------------------------------------------------------------
# STEP 3: APPLY GAUSSIAN BLUR
# ------------------------------------------------------------

blur = cv2.GaussianBlur(gray, (5, 5), 0)

# ------------------------------------------------------------
# STEP 4: EDGE DETECTION USING CANNY
# ------------------------------------------------------------

edges = cv2.Canny(blur, 50, 150)

# ------------------------------------------------------------
# STEP 5: HOUGH LINE TRANSFORM
# ------------------------------------------------------------

lines = cv2.HoughLinesP(
    edges,
    rho=1,
    theta=np.pi / 180,
    threshold=80,
    minLineLength=80,
    maxLineGap=10
)

line_image = img.copy()

detected_lines = []

if lines is not None:

    for i, line in enumerate(lines):

        x1, y1, x2, y2 = line # Fix: Removed [0] as 'line' is already the array of coordinates

        # Draw detected line
        cv2.line(
            line_image,
```

```
            (x1, y1),
            (x2, y2),
            (0, 255, 0),
            3
        )

        # Calculate line length
        length = np.sqrt(
            (x2 - x1) ** 2 +
            (y2 - y1) ** 2
        )

        # Calculate angle
        angle = np.degrees(
            np.arctan2(y2 - y1, x2 - x1)
        )

        detected_lines.append([
            i + 1,
            x1, y1,
            x2, y2,
            round(length, 2),
            round(angle, 2)
        ])

# ------------------------------------------------------------
# STEP 6: DISPLAY LINE PARAMETERS
# ------------------------------------------------------------

line_columns = [
    "Line",
    "x1", "y1",
    "x2", "y2",
    "Length",
    "Angle"
]

line_df = pd.DataFrame(
    detected_lines,
    columns=line_columns
)

print("\n========== DETECTED LINES ==========")

if len(line_df) > 0:
    print(line_df.to_string(index=False))
else:
    print("No lines detected.")

print("\nTotal Lines Detected:",
      len(detected_lines))

# ------------------------------------------------------------
# STEP 7: HOUGH CIRCLE TRANSFORM
# ------------------------------------------------------------

circles = cv2.HoughCircles(
    gray,
    cv2.HOUGH_GRADIENT,
    dp=1.2,
    minDist=50,
    param1=100,
    param2=30,
    minRadius=20,
    maxRadius=100
)
```

```
circle_image = img.copy()

detected_circles = []

if circles is not None:

    circles = np.uint16(
        np.around(circles)
    )

    for i, circle in enumerate(circles[0, :]):

        x, y, r = circle

        # Draw circle
        cv2.circle(
            circle_image,
            (x, y),
            r,
            (0, 255, 0),
            3
        )

        # Draw center
        cv2.circle(
            circle_image,
            (x, y),
            4,
            (0, 0, 255),
            -1
        )

        detected_circles.append([
            i + 1,
            int(x),
            int(y),
            int(r)
        ])

# ------------------------------------------------------------
# STEP 8: DISPLAY CIRCLE PARAMETERS
# ------------------------------------------------------------

circle_columns = [
    "Circle",
    "Center X",
    "Center Y",
    "Radius"
]

circle_df = pd.DataFrame(
    detected_circles,
    columns=circle_columns
)

print("\n========== DETECTED CIRCLES ==========")

if len(circle_df) > 0:
    print(circle_df.to_string(index=False))
else:
    print("No circles detected.")

print("\nTotal Circles Detected:",
      len(detected_circles))

# ------------------------------------------------------------
# STEP 9: DISPLAY ALL PROCESSING STAGES
```

```
# ------------------------------------------------------------

plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(
    cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
)
plt.title("1. Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("2. Grayscale")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(blur, cmap="gray")
plt.title("3. Gaussian Blur")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(edges, cmap="gray")
plt.title("4. Canny Edge Detection")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(
    cv2.cvtColor(line_image, cv2.COLOR_BGR2RGB)
)
plt.title("5. Hough Line Detection")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(
    cv2.cvtColor(circle_image, cv2.COLOR_BGR2RGB)
)
plt.title("6. Hough Circle Detection")
plt.axis("off")

plt.tight_layout()
plt.show()

# ------------------------------------------------------------
# STEP 10: SAVE RESULTS
# ------------------------------------------------------------

cv2.imwrite(
    "hough_line_detection.jpg",
    line_image
)

cv2.imwrite(
    "hough_circle_detection.jpg",
    circle_image
)

print("\nResults saved successfully.")
```

```
========== DETECTED LINES ==========
 Line   x1   y1   x2   y2  Length   Angle
    1   53   97  597   97  544.00    0.00
    2  106  102  601  102  495.00    0.00
    3  150  397  599  148  513.42  -29.01
    4   97  447   97   53  394.00  -90.00
    5  150  402  601  152  515.66  -29.00
    6  102  451  102  106  345.00  -90.00

Total Lines Detected: 6

========== DETECTED CIRCLES ==========
 Circle  Center X  Center Y  Radius
      1       249       249      73
      2       501       351      46

Total Circles Detected: 2
```




```
Results saved successfully.
```